

Course organisation

Probability in Computer Science (M. Češka)

- 1st - 3rd week: lectures and numerical tutorials
- 1st test (10 pts) in the 4th week (Thursday 9.10 at 18:00)

Advanced Statistics (P. Hrabec)

- 4th - 13th week: lectures and numerical tutorials
- 2nd test (10 pts) in the 10th week (Thursday 20.11 at 18:00)

2. seminars (demo-cvičení)

- 5th week (Thursday 16.10 at 18:00): Application of basic statistical methods, statistical programming (V. Mrázek)
- 11th week (Thursday 27.11 at 18:00): Application of advanced statistical methods (P. Hrabec)

2 projects: 8+12 pts (at least 2+4 pts)

- Basic statistics and programming (V. Mrázek, deadline 7th week)
- Advanced statistics (P. Hrabec, deadline 13th week)

Accreditation (zápočet): at least 20 pts (out of 40)

Final exam: 60 pts (at least 25 pts)

Probability in Computer Science

Milan Češka and Roman Andriushchenko

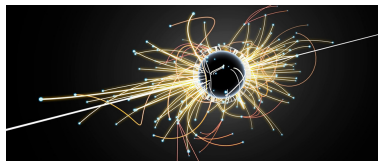
MSP 2025

Does the universe have true randomness?



Newtonian physics suggests that the universe evolves deterministically.

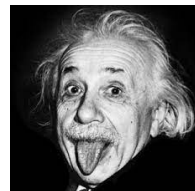
Quantum physics says otherwise.



Does the universe have true randomness?

God does not play dice with the world.

– *Albert Einstein*



Einstein, don't tell God what to do.

– *Niels Bohr*

Does the universe have true randomness?

Even if it does not, we can still model our uncertainty about things using probability.

Randomness is an essential tool for modelling and analysing nature as well as artificial systems.

Randomness plays a key role in computer science and algorithmics.

Probability in computer science

- **to model uncertainty**
 - dependability wrt. uncertain system components
 - planning wrt. unpredictable environment
- randomisation
 - **randomised algorithms**
 - symmetry breaking (distributed computing)
 - cryptography
- performance evaluation
- stochastic control theory
- probabilistic programming – inference from data
- computational system biology

Probability in computer science

Article

Mastering Atari, Go, chess and shogi by planning with a learned model

<https://doi.org/10.1038/s41586-020-03051-4>

Received: 3 April 2020

Accepted: 7 October 2020

Published online: 23 December 2020

Check for updates

Julian Schrittwieser¹, Ioannis Antonoglou^{1,2,3}, Thomas Hubert^{1,4}, Karen Simonyan¹, Laurent Sifre¹, Simon Schmitt¹, Arthur Guez¹, Edward Lockhart¹, Demis Hassabis¹, Thore Graepel^{1,5}, Timothy Lillicrap¹ & David Silver^{1,2,3,6,7}

Constructing agents with planning capabilities has long been one of the main challenges in the pursuit of artificial intelligence. Tree-based planning methods have enjoyed huge success in challenging domains, such as chess¹ and Go², where a perfect simulator is available. However, in real-world problems, the dynamics governing the environment are often complex and unknown. Here we present the MuZero algorithm, which, by combining a tree-based search with a learned model, achieves superhuman performance in a range of challenging and visually complex domains, without any knowledge of their underlying dynamics. The MuZero algorithm learns an iterable model that produces predictions relevant to planning: the action selection policy, the value function and the reward. When evaluated on 57 different Atari games³—the canonical video game environment for testing artificial intelligence techniques, in which model-based planning approaches have historically struggled⁴—the MuZero algorithm achieved state-of-the-art performance. When evaluated on Go, chess and shogi—canonical environments for high-performance planning—the superhuman performance of the AlphaZero algorithm⁵ that was supplied with the rules of the game.

Planning algorithms based on lookahead search have achieved remarkable successes in artificial intelligence. Human world champions have been defeated in classic games such as checkers⁶, chess⁷, Go⁸ and poker⁹, and planning algorithms have had real-world impact in applications ranging from logistics¹⁰ to chemical synthesis¹¹. However, these planning algorithms all rely on knowledge of the environment's dynamics, such as the rules of the game or an accurate simulator, preventing their direct application to real-world domains such as robotics, industrial control or intelligent assistants, where the dynamics are normally unknown.

Model-based reinforcement learning (RL)¹² aims to address this issue by first learning a model of the environment's dynamics and then planning with respect to the learned model. Typically, these models have either focused on reconstructing the true environmental state^{13–14} or the sequence of full observations^{15–19}. However, previous work^{20–22} remains far from the state of the art in visually rich domains, such as Atari 2600 games²³. Instead, the most successful methods are based on model-free RL^{24–26}, that is, they estimate the optimal policy and/or value function directly from interactions with the environment. However, model-free algorithms are in turn far from the state of the art in domains that require precise and sophisticated lookahead, such as chess and Go.

Here we introduce MuZero, a new approach to model-based RL that achieves both state-of-the-art performance in Atari 2600 games—a visually complex set of domains—and superhuman performance in precision planning tasks such as chess, shogi and Go, without prior

knowledge of the game dynamics. MuZero builds on AlphaZero's⁵ powerful search and policy iteration algorithms, but incorporates a learned model into the training procedure. MuZero also extends AlphaZero to a broader set of environments, including single-agent domains and non-zero-rewards at intermediate time steps.

The main idea of the algorithm (summarized in Fig. 1) is to predict those aspects of the future that are directly relevant for planning. The model receives the observation (for example, an image of the Go board or the Atari screen) as an input and transforms it into an hidden state. The hidden state is denuded iteratively by recurrent processes that receives the previous hidden state and a hypothetical next action. At every one of these steps, the model produces a policy (predicting the move to play), a value function (predicting the cumulative reward, for example, the eventual winner) and immediate reward prediction (for example, the points scored by playing a move). The model is trained end-to-end, with the objective of accurately simulating these three important quantities, to match the improved policy and value function generated by search, as well as the observed reward. There is no direct requirement or constraint on the hidden state to capture all information necessary to reconstruct the original observation, drastically reducing the amount of information the model has to maintain and predict. Neither is there any requirement for the hidden state to match the unknown, true state of the environment; nor any other constraints on the semantics of state. Instead, the hidden states are free to represent any state that correctly estimates the policy, value function and

detail. None of these previous methods have constructed a model that facilitates effective planning in visually complex domains such as Atari; results lag behind well-tuned, model-free RL methods, even in terms of data efficiency²⁷.

A quite different approach to model-based RL has recently been developed, focused not on predicting the value function^{28–30}.

The main idea of these methods is to construct an abstract MDP model

MuZero algorithm

We now describe the MuZero algorithm in more detail. Predictions are made at each time step t , for each $k \in \{0, \dots, K\}$ steps, by a model μ_k with parameters θ_k conditioned on past observations $o_0, \dots, o_{t-1}$ and $k > 0$ on future actions $a_0, \dots, a_{t-1}$. The model predicts three future quantities: the policy $\pi_k = \pi(\theta_k; a_0, \dots, a_{t-1}, o_0, \dots, o_{t-1})$ the value

Nature | Vol 588 | 24(31 December 2020 | 605

Article

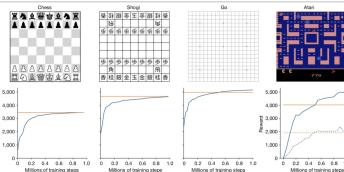


Fig. 1 | Evaluation of MuZero on Atari, Go, Chess, and Shogi. The figure shows four panels: Chess, Shogi, Go, and Atari. Each panel displays a game board and a line graph of 'Reward' vs 'Millions of training steps'. The Atari panel shows a game screen. The graphs show that MuZero (red line) outperforms other methods (blue and green lines) across all domains.

across all 57 games are shown on the y-axis. The scores for MuZero (the previous state of the art in the domain, based on model-free RL) are indicated by the horizontal orange lines. Performance in Atari was evaluated using 500 simulations per move for the top players. AlphaZero (A) is indicated by the blue line and AlphaZero (S) is indicated by the horizontal orange line. For Atari, we use an (full line) and an (dashed line) line for normalized scores.

function $v_k = \mathbb{E}[R_{t+k} | s_t, a_0, \dots, a_{t-1}, o_0, \dots, o_{t-1}]$ and, for $k > 0$, also the immediate reward $r_k = R_{t+k}$, where t is time, observed policy π_k is the policy used to select actions and γ is the discount factor.

Internally, at each time step (substeps are suppressed for simplicity), the models are represented by the combination of a representation function, a recurrent process and a prediction function. The dynamics function, a recurrent process, $\phi_k: s^{(k)} \rightarrow s^{(k+1)}$, that computes, at each hypothetical step, a hidden state $s^{(k)}$ and an internal state $z^{(k)}$. It mirrors the structure of an MDP model that computes the expected reward and state transition for a given state and action³¹. However, unlike traditional approaches to model-based RL³², this internal state $z^{(k)}$ has no semantics of environment state attached to it—its simple hidden state of the overall model and its sole purpose is to accurately predict relevant facts, future quantities, policies, values and reward. In this paper, the dynamics function is represented deterministically; the extension to stochastic transitions is left for future work. A prediction function f_k computes the policy and value functions from the internal state $s^{(k)}, z^{(k)}$. f_k is also the joint policy and value network of AlphaZero. A representation function h_k initializes the 'root' state $s^{(0)}$ by encoding past observations, $o_0, \dots, o_{t-1}$. Again, this has no special semantics beyond its support for future predictions.

Given such a model, it is possible to search over hypothetical future trajectories $\tau_k = \{s^{(k)}, a^{(k)}, z^{(k)}\}_{k=0}^{K-1}$ given past observations $o_0, \dots, o_{t-1}$. For example, a naive search could simply select the k step action sequence that maximizes the value function. More generally, we may apply any MDP planning algorithm to the internal rewards and state space induced by the dynamics function. Specifically, we use an MCTS algorithm similar to AlphaZero's search, generalized to allow for single-agent domains and intermediate rewards (Methods). The MCTS algorithm may be viewed as a search policy $\pi_k = \pi(\theta_k; a_0, \dots, a_{t-1}, o_0, \dots, o_{t-1})$ and a value network $v_k = v(\theta_k; a_0, \dots, a_{t-1}, o_0, \dots, o_{t-1})$ in both steps action and prediction.

by the current model parameters θ_k and combines these values together using lookahead search to produce an improved policy π_k and improved value function v_k at the root of the search tree. The next action $a_t = \pi_k(s_t)$ is then chosen by the search policy.

All parameters of the model are trained jointly to accurately match the policy, value function and reward prediction, for every hypothetical step k , so that the corresponding quantities observed after a usual time steps have policy π_k . Also like AlphaZero, value targets are generated by playing out the game or MDP using the search policy. However, unlike AlphaZero, we allow for long episodes with discounting and intermediate rewards by computing an intermediate z_t that bootstraps n steps into the future from the search value, $z_t = \gamma^n v_{t+n} + (1 - \gamma^n) r_{t+n}$. Final outcomes lose, draw, win in board games are treated as reward $z_t \in \{-1, 0, 1\}$ (occurring at the final step of the episode). Specifically, the second objective to minimize the error between the value function v_k and the value target, z_t . The third objective is to minimize the error between the predicted immediate reward r_k and the observed immediate reward r_t . Finally, an L2 regularization loss is added, scaled by a constant λ , leading to the overall loss

$$L(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \mathbb{E}_{\pi_k} [r_k - v_k]^2 + \frac{\lambda}{K} \sum_{k=0}^{K-1} \mathbb{E}_{\pi_k} [r_k - v_k]^2 + \frac{\lambda}{K} \sum_{k=0}^{K-1} \mathbb{E}_{\pi_k} [r_k - v_k]^2. \quad (1)$$

where π_k and f_k are loss functions for policy and value, respectively. Supplementary Fig. 2 summarizes the equations governing how the MuZero algorithm plans, acts, learns and rewards. We note that for chess, Go and shogi, the same squared error loss as AlphaZero is used for reward and value. A cross-entropy loss was found to be more stable than a squared error when reconstructing rewards and values

DeepMind, London, UK; ²University College London, London, UK. These authors contributed equally: Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, David Silver. *email: david.silver@google.com

Outline

Part 1: Modelling and Analysis of Probabilistic Systems

- Lecture 1: Probability theory recap, Markov chains
- Lecture 2: Markov decision processes

These lectures are partially based on material from Chapter 10 of “*Principles of Model Checking*” by Christel Baier and Joost-Pieter Katoen (MIT Press) and “*Stochastic Model Checking*” by Marta Kwiatkowska, Gethin Norman and David Parker.

Part 2: Introduction to Randomised Algorithms

- Lecture 3: Probabilistic analysis of programs and basic randomised algorithms

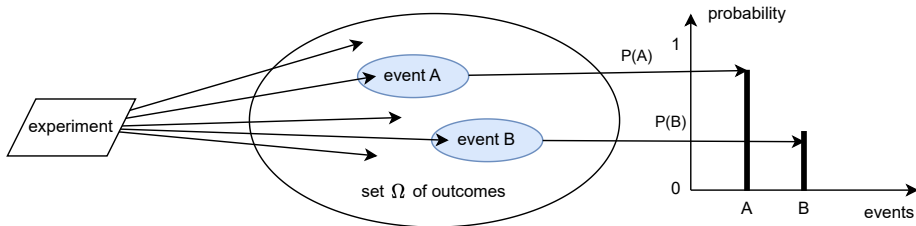
This lecture is partially based on material from Chapter 5 of “*Introduction to Algorithms (3rd edition)*” by Thomas Cormen et al. (MIT Press).

- Three numerical practicals

Probability space - motivation

probability space = mathematical description of an uncertain situation

- Ω is the *sample space* – the set of all possible outcomes of an experiment
- subsets of Ω are *events*
- *probability measure* P assigns probabilities to each event



Probability space – definition

Definition (probability space)

Probability space is a triple $(\Omega, \mathcal{A}, P)$, where

- Ω is a non-empty *sample space*
- $\mathcal{A} \subseteq 2^\Omega$ is the σ -algebra of *events* – events are closed under complement and countable unions, Ω itself is an event
- $P : \mathcal{A} \rightarrow [0, 1]$ is the *probability measure* satisfying
 - $P(\Omega) = 1$
 - if $\{A_i\}_{i=1}^\infty \subseteq \mathcal{A}$ is a countable collection of pairwise disjoint events, then $P(\bigcup_{i=1}^\infty A_i) = \sum_{i=1}^\infty P(A_i)$

Probability space - example

Example. Assume a roll of a fair 3-sided dice.

- $\Omega = \{1, 2, 3\}$
- $\mathcal{A} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$
- $P(A) = \frac{|A|}{3}$, e.g. $P(\{1, 2\}) = \frac{2}{3}$

Note: there are many other σ -algebras on 2^Ω , e.g. here is σ -algebra that speaks only whether the outcome was odd or even:

$$\mathcal{A}_2 = \{\emptyset, \{1, 3\}, \{2\}, \{1, 2, 3\}\}.$$

Probability space - basic properties

Let $(\Omega, \mathcal{A}, P)$ be a probability space and let $A, B \in \mathcal{A}$ be events.

- $P(\emptyset) = 0$
- $P(A) = 1 - P(\Omega \setminus A)$
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- $A \cap B = \emptyset \Rightarrow P(A \cup B) = P(A) + P(B)$
- $A \subseteq B \Rightarrow P(A) \leq P(B)$

Alternative notation: $P(A, B) := P(A \cap B)$

Conditional probability

Definition

Let $(\Omega, \mathcal{A}, P)$ be the probability space, $A, B \in \mathcal{A}$ and $P(B) > 0$. Then

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

is the *conditional probability* of event A given B .

Example. Assume a throw of a 6-sided dice landed on an even number. Then, the probability that it was 1,2 or 3 is:

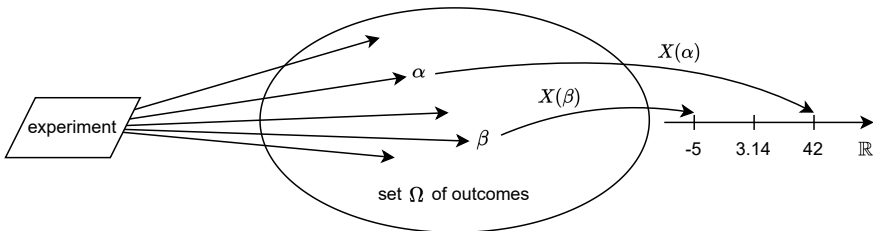
$$P(\{1, 2, 3\} \mid \{2, 4, 6\}) = \frac{P(\{1, 2, 3\} \cap \{2, 4, 6\})}{P(\{2, 4, 6\})} = \frac{P(\{2\})}{P(\{2, 4, 6\})} = \frac{1/6}{1/2} = 1/3$$

Definition

Events A, B are *independent* if $P(A \cap B) = P(A) \cdot P(B)$.

Random variables – motivation

- quite often, the outcome of an experiment is a numerical value
 - e.g. room temperature, stock price, network latency. . .
- in other cases, we may associate a number with the experiment outcome
 - e.g. if a student in this room is picked randomly, we may record their height, blood pressure, number of siblings. . .
- **random variable** X is a function that associates experiment outcome ω with a real number $X(\omega)$



Random variables

Definition

Let $(\Omega, \mathcal{A}, P)$ be the probability space. A *random variable* (r.v.) is a function $X: \Omega \rightarrow \mathbb{R}$ s.t. for each $x \in \mathbb{R}$: $\{\omega \in \Omega \mid X(\omega) \leq x\} \in \mathcal{A}$.

- notation: $P(X = x) := P(\{\omega \in \Omega \mid X(\omega) = x\})$
 - and similarly $P(X \leq x)$, $P(X > x)$ etc.
- *support* of X is the set $\{x \in \mathbb{R} \mid P(X = x) > 0\}$ of possible values of X

Example. Assume two throws of a fair coin and let X be the r.v. denoting the number of observed heads. Support of X is $\{0, 1, 2\}$, where

$$P(X = 0) = 1/4 \quad P(X = 1) = 1/2 \quad P(X = 2) = 1/4$$

Distribution function

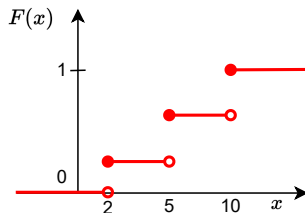
- CDF is a real-valued function that can perfectly describe an arbitrary random variable

Definition (CDF)

Cumulative distribution function (CDF, distribuční funkce) of a random variable X is a function $F: \mathbb{R} \rightarrow [0, 1]$ where $F(x) = P(X \leq x)$.

Properties:

- since r.v. X satisfies $\forall x \in \mathbb{R}: \{\omega \in \Omega \mid X(\omega) \leq x\} \in \mathcal{A}$, then $F(x)$ is defined for every x
- F is non-decreasing and right-continuous
- $\lim_{x \rightarrow -\infty} F(x) = 0, \lim_{x \rightarrow \infty} F(x) = 1$
- $P(X = x) = F(x) - \lim_{t \rightarrow x^-} F(t)$



Definition

If the support of a random variable X is finite/countable, we say that X is a *discrete random variable*, otherwise X is *continuous*.

- PMF is the most intuitive way to describe a discrete r.v.

Definition

Probability mass function (PMF, pravděpodobnostní funkce) of a discrete random variable X is

$$p(x) = P(X = x).$$

Properties:

- $\sum_x p(x) = 1$
- $F(x) = P(X \leq x) = \sum_{t \leq x} p(t)$

Characteristics of a random variable

Definition

Let X be a discrete random variable. *Expectation* of X is

$$E[X] = \sum_x x \cdot p(x).$$

Assume $a, b \in \mathbb{R}$ and X, Y are random variables

- $E[aX + b] = a \cdot E[X] + b$
- $E[X + Y] = E[X] + E[Y]$

Characteristics of a random variable

Definition

Let X be a discrete random variable. *Variance* of X is

$$\text{Var}(X) = \mathbb{E} \left[(X - \mathbb{E}[X])^2 \right] = \sum_x (x - \mathbb{E}[X])^2 \cdot p(x).$$

Standard deviation of X is

$$\sigma(X) = \sqrt{\text{Var}(X)}.$$

Assume $a, b \in \mathbb{R}$ and X, Y are random variables

- $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$
- $\text{Var}(aX + b) = a^2 \cdot \text{Var}(X)$
- $\text{Var}(X \pm Y) = \text{Var}(X) + \text{Var}(Y)$ if X & Y are independent

Notable discrete distributions

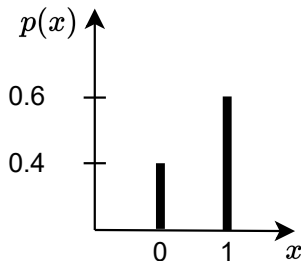
Bernoulli random variable $X \sim A(p)$ describes the binary outcome (0/1) of an experiment that has probability p of “success” (outcome 1). Also known as *indicator* r.v.

- PMF:

$$p(x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \\ 0 & \text{otherwise} \end{cases}$$

- $E[X] = p$

- $\text{Var}(X) = p(1 - p)$



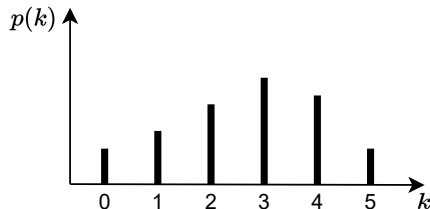
$A(0.6)$

Examples: whether a coin toss resulted in heads, whether a stock goes up, whether a patient has been diagnosed with a disease, etc.

Notable discrete distributions

Binomial random variable $X \sim Bi(n, p)$, describes the number of successes of n consecutive independent Bernoulli trials, where each trial has probability p of success: $X = \sum_{i=1}^n X_i$, $X_i \sim A(p)$.

- PMF: $p(k) = \binom{n}{k} p^k (1-p)^{n-k}$,
 $k = 0, 1, \dots, n$
- $E[X] = np$
- $\text{Var}(X) = np(1-p)$



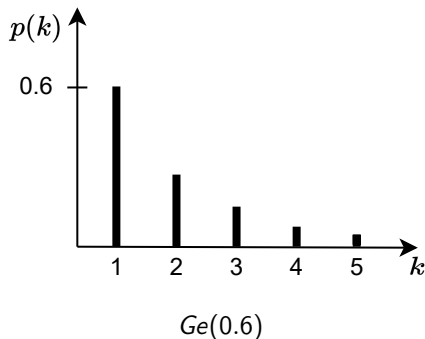
$Bi(5, 0.6)$

Examples: number of faulty components found in a production batch, number of students that passed MSP, number of network packets that reached destination, etc.

Notable discrete distributions

Geometric random variable $X \sim \text{Ge}(p)$ describes the number of Bernoulli trials performed until the first success.

- PMF: $p(k) = (1 - p)^{k-1}p$,
 $k = 1, 2, \dots$
- $E[X] = \frac{1}{p}$
- $\text{Var}(X) = \frac{1-p}{p^2}$



Examples: number of lottery tickets bought until the first win, number of attempts until a phone connection is established, number of job interviews conducted until a hire, etc.

Notable discrete distributions

Multinomial random vector (generalised binomial) $(X_1, \dots, X_k)^T \sim Mu(n, p_1, \dots, p_k)$ describes experiment consisting of n consecutive independent trials, where each trial has k outcomes with probabilities $p_1, \dots, p_k$, $\sum_{i=1}^k p_i = 1$. Random variable X_i then describes the number the i -th outcome was observed.

- PMF: $p(n_1, n_2, \dots, n_k) = \frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!} p_1^{n_1} \cdot p_2^{n_2} \cdot \dots \cdot p_k^{n_k}$, $\sum_{i=1}^k n_i = n$
- $E[X_i] = np_i$
- $\text{Var}(X_i) = np_i(1 - p_i)$

Examples: number of each of outcomes of several dice rolls, number of votes received by each party in an election, number of images classified as cat/dog/neither by a neural network, etc.

Probabilistic/stochastic processes

Random variables that evolve over time

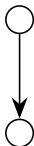
	fully probabilistic	nondeterministic
discrete time	discrete-time Markov chains (DTMCs)	Markov decision processes (MDPs)
continuous time	continuous-time Markov chains (CTMCs)	CTMDPs/IMCs , probabilistic timed automata (PTAs)

Other important characteristics:

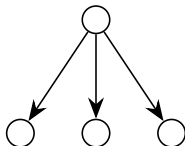
- **finite** vs countably infinite vs uncountably infinite
- **fully observable** vs parametric vs partially observable
- labelled transition systems

System behaviour for a given input

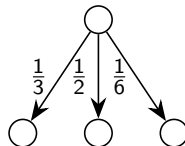
- How to model the system behaviour for a given input?



deterministic system

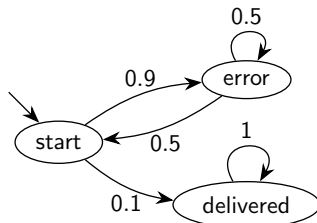


transition system
(nondeterministic branching)



Markov chain
(probabilistic branching)

- Markov chain = transition system with probabilistic distribution over successors
- example of a communication protocol:

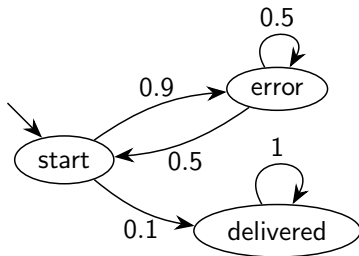


Discrete-time Markov chains

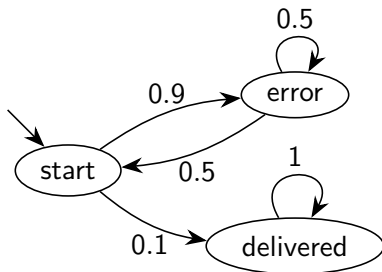
Definition (DTMC)

A *discrete-time Markov chain (DTMC)* is a tuple $(S, s_0, \mathbf{P})$, where

- S is a finite set of states (*state space*)
- $s_0 \in S$ is the *initial state*
- $\mathbf{P} : S \times S \rightarrow [0, 1]$ is the *transition probability matrix*, where
 $\forall s \in S : \sum_{s' \in S} \mathbf{P}(s, s') = 1$



Transition probability matrix

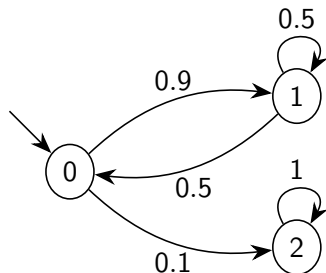


$$P = \begin{pmatrix} 0 & 0.9 & 0.1 \\ 0.5 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- P is a (right) stochastic matrix: rows of P sum to 1
- transition $P(s, s) > 0$ from state into itself is called a *self-loop*
- state $s \in S$ for which $P(s, s) = 1$ is called *absorbing*

Memorylessness

- assume each state is associated with a unique number: $S \subset \mathbb{R}$
- let $X(k) \in S$ be the random variable describing the state of the DTMC at time $k \geq 0$



- *memorylessness (Markov property):*

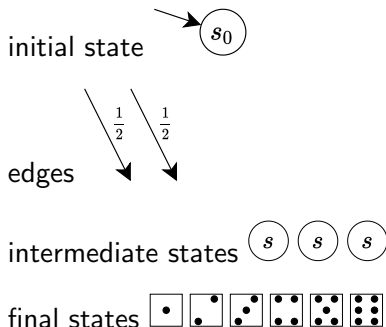
$$P(X(k+1)=s_{k+1} \mid X(k)=s_k, \dots, X(0)=s_0) = P(X(k+1)=s_{k+1} \mid X(k)=s_k)$$

i.e., the probability of the transition depends only on the current state and is independent of the past

Example: Knuth-Yao dice

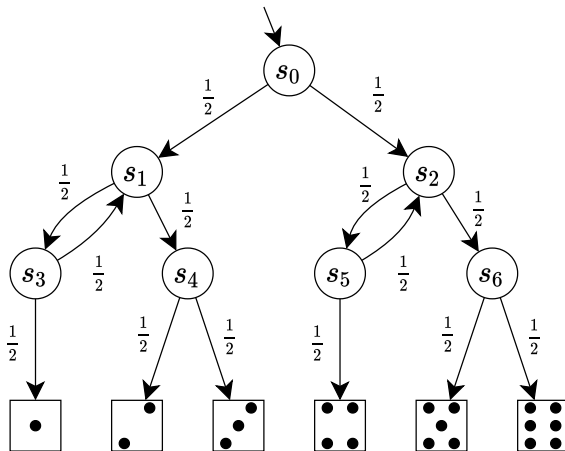
Design a protocol simulating a fair dice using a fair coin

- ingredients:



- probability of each outcome must be $\frac{1}{6}$
- final states are absorbing

Example: Knuth-Yao dice (solution)



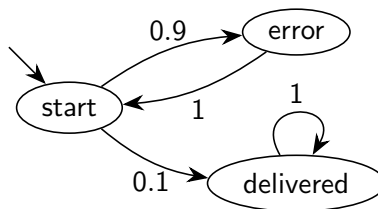
Paths in DTMCs

- a *path* is a sequence $s_0 s_1 s_2 \dots$ of states such that $P(s_i, s_{i+1}) > 0, \forall i$
- path = one possible execution of the system
- probability of a *finite* path $s_0 s_1 s_2 \dots s_n$:

$$P(s_0 s_1 s_2 \dots s_n) = P(s_0, s_1) \cdot P(s_1, s_2) \cdot \dots \cdot P(s_{n-1}, s_n)$$

- we can multiply transition probabilities due to memorylessness (individual transitions are independent)
- probability of an *infinite* path $s_0 s_1 s_2 \dots$ is nonzero if the path uses only a finite number of uncertain transitions, i.e., after some step n , $P(s_i, s_{i+1}) = 1$ for all $i > n$

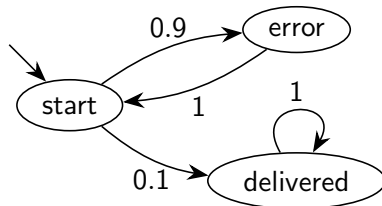
Paths – example



- probability of delivering a message *within three steps*:

$$P(\text{start delivered}) + P(\text{start error start delivered}) = \\ 0.1 + 0.9 \cdot 1 \cdot 0.1 = 0.19$$

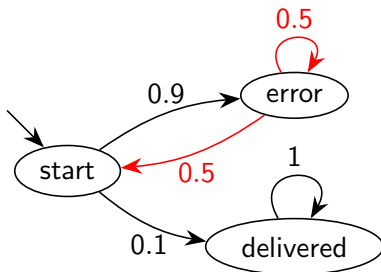
Paths – example



- probability of *eventually* delivering a message:

$$\begin{aligned}
 \sum_{n=0} P((\text{start error})^n \text{ start delivered}) &= \sum_{n=0} (0.9 \cdot 1)^n \cdot 0.1 \\
 &= 0.1 \cdot \sum_{n=0} 0.9^n = 0.1 \cdot \frac{1}{1 - 0.9} = 1
 \end{aligned}$$

Paths – example

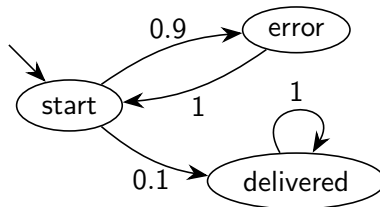


- probability of *eventually* delivering a message:

$$\begin{aligned}\sum_{n=0} P((\text{start error})^n \text{ start delivered}) &= \sum_{n=0} (0.9 \cdot 1)^n \cdot 0.1 \\ &= 0.1 \cdot \sum_{n=0} 0.9^n = 0.1 \cdot \frac{1}{1 - 0.9} = 1\end{aligned}$$

- does the probability change?

Paths – example



- $P((\text{start error})^\omega) = (0.9 \cdot 1)^\omega = 0$
- $P((\text{start error})^n \text{ start delivered}^\omega) = (0.9 \cdot 1)^n \cdot 0.1 \cdot 1^\omega = 0.9^n \cdot 0.1 > 0$

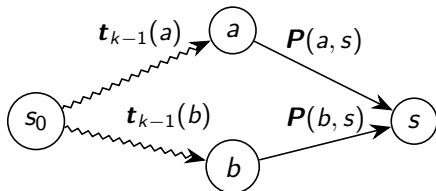
Transient analysis

How does the system evolve in time (after starting in s_0)?

- *transient probability*: $t_k(s) = P(X(k) = s \mid X(0) = s_0)$
 - the probability of being in state $s \in S$ at time $k \geq 0$
- sum probabilities of all paths of length k leading to s (expensive)
- apply the Markov property directly:

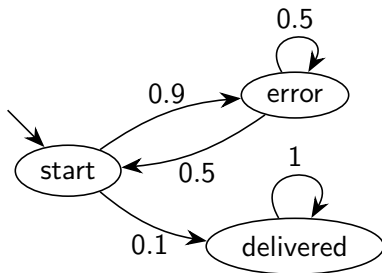
$$t_k(s) = \sum_{s' \in S} t_{k-1}(s') \cdot P(s', s) \text{ where } t_0(s_0) = 1$$

- forward propagation of the probability
- corresponds to a chain of vector-matrix multiplications:
 $t_k = t_{k-1} \cdot P$ (t 's are row vectors)



$$t_k(s) = t_{k-1}(a) \cdot P(a, s) + t_{k-1}(b) \cdot P(b, s)$$

Transient analysis – example



$$P = \begin{pmatrix} 0 & 0.9 & 0.1 \\ 0.5 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{t}_0 = (1, 0, 0)$$

$$\mathbf{t}_1 = \mathbf{t}_0 \cdot \mathbf{P} = (0, 0.9, 0.1)$$

$$\mathbf{t}_2 = \mathbf{t}_1 \cdot \mathbf{P} = (0.45, 0.45, 0.1)$$

$$\mathbf{t}_3 = \mathbf{t}_2 \cdot \mathbf{P} = (0.225, 0.63, 0.145)$$

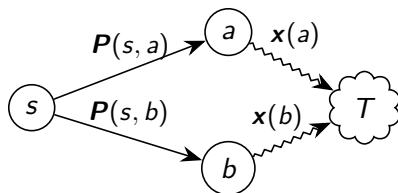
$$\mathbf{t}_4 = \mathbf{t}_3 \cdot \mathbf{P} = (0.315, 0.5175, 0.1675)$$

$$\mathbf{t}_5 = \mathbf{t}_4 \cdot \mathbf{P} = (0.25875, 0.54225, 0.199)$$

Reachability

- assume a nonempty set $T \subseteq S$ of *target states*
- what's the probability $\mathbf{x}(s) := P(s \rightarrow T)$ of eventually reaching T from an arbitrary state $s \in S$?
- solve the following linear system:

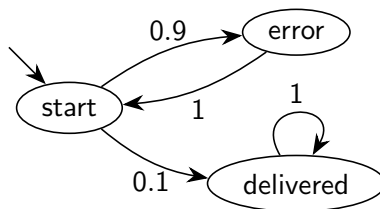
$$\begin{aligned}\mathbf{x}(s) &= 1 && \text{if } s \in T \\ \mathbf{x}(s) &= \sum_{s' \in S} P(s, s') \cdot \mathbf{x}(s') && \text{if } s \notin T\end{aligned}$$



$$\mathbf{x}(s) = P(s, a) \cdot \mathbf{x}(a) + P(s, b) \cdot \mathbf{x}(b)$$

- unfortunately, such system might not have a unique solution ...

Reachability – uniqueness



- $T = \{error\}$

$$\mathbf{x}(start) = 0.9 \cdot \mathbf{x}(error) + 0.1 \cdot \mathbf{x}(delivered)$$

$$\mathbf{x}(error) = 1 \quad (error \in T)$$

$$\mathbf{x}(delivered) = 1 \cdot \mathbf{x}(delivered)$$

$$\Rightarrow \mathbf{x} = (0.9 + 0.1 \cdot p, 1, p)$$

Reachability - adding S_0 states

- denote $S_0 = \{s \in S \mid T \text{ is not reachable from } s\}$
- the reachability probabilities $\mathbf{x}(s)$ are given as a solution of the following linear system:

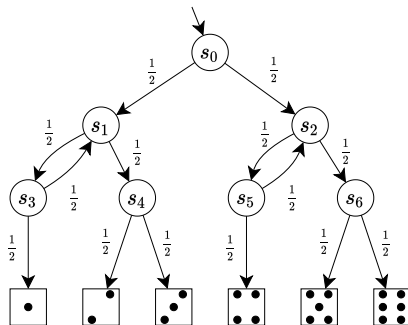
$$\mathbf{x}(s) = 1, \quad \text{if } s \in T$$

$$\mathbf{x}(s) = 0, \quad \text{if } s \in S_0$$

$$\mathbf{x}(s) = \sum_{s' \in S} \mathbf{P}(s, s') \cdot \mathbf{x}(s'), \quad \text{if } s \notin T \cup S_0$$

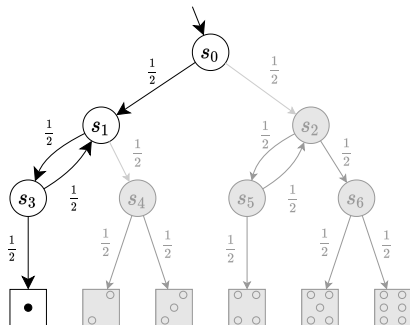
Reachability – example

- compute $x(s) = P(s \rightarrow T)$ for $T = \{\square\}$
 - $x(\square) = 1$



Reachability – example

- compute $x(s) = P(s \rightarrow T)$ for $T = \{\Box\}$
 - $x(\Box) = 1$
- $S_0 = \{s_2, s_4, s_5, s_6, \Box, \Box, \Box, \Box, \Box\}$
 - $x(s) = 0$ for $s \in S_0$



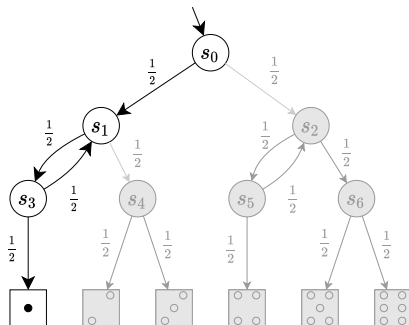
Reachability – example

- compute $\mathbf{x}(s) = P(s \rightarrow T)$ for $T = \{\square\}$
 - $\mathbf{x}(\square) = 1$
- $S_0 = \{s_2, s_4, s_5, s_6, \square, \square, \square, \square, \square\}$
 - $\mathbf{x}(s) = 0$ for $s \in S_0$
- for $s \notin T \cup S_0$, solve the linear system:

$$\mathbf{x}(s_0) = \frac{1}{2} \cdot \mathbf{x}(s_1)$$

$$\mathbf{x}(s_1) = \frac{1}{2} \cdot \mathbf{x}(s_3)$$

$$\mathbf{x}(s_3) = \frac{1}{2} \cdot \mathbf{x}(\square) + \frac{1}{2} \cdot \mathbf{x}(s_1)$$



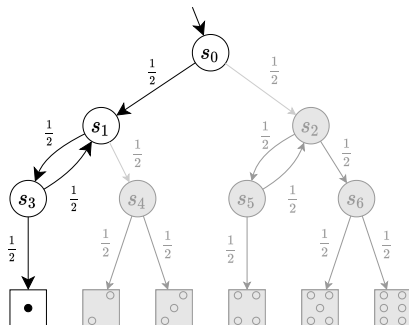
Reachability – example

- compute $\mathbf{x}(s) = P(s \rightarrow T)$ for $T = \{\square\}$
 - $\mathbf{x}(\square) = 1$
- $S_0 = \{s_2, s_4, s_5, s_6, \square, \square, \square, \square, \square\}$
 - $\mathbf{x}(s) = 0$ for $s \in S_0$
- for $s \notin T \cup S_0$, solve the linear system:

$$\mathbf{x}(s_0) = \frac{1}{2} \cdot \mathbf{x}(s_1) = \frac{1}{6}$$

$$\mathbf{x}(s_1) = \frac{1}{2} \cdot \mathbf{x}(s_3) = \frac{1}{3}$$

$$\mathbf{x}(s_3) = \frac{1}{2} \cdot \mathbf{x}(\square) + \frac{1}{2} \cdot \mathbf{x}(s_1) = \frac{2}{3}$$



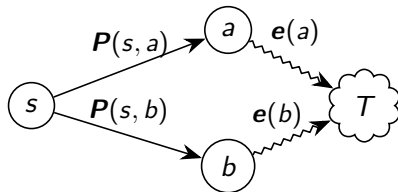
Expected time to reach a state

- Given the set $T \subseteq S$ of target states, let $K(s)$ be the random variable denoting the time (number of steps) required to reach T from state s .
- Note that $K(s)$ is defined only if the set T is reachable from s with probability 1: $P(s \rightarrow T) = 1$
- Obtaining PMF of $K(s)$ is hard, but it is surprisingly easy to compute its expectation $e(s) := E[K(s)]$:

$$e(s) = 0$$

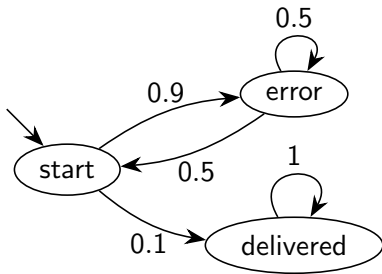
$$e(s) = 1 + \sum_{s' \in S} P(s, s') \cdot e(s') \quad \text{if } s \notin T$$

if $s \in T$



$$e(s) = 1 + P(s, a) \cdot e(a) + P(s, b) \cdot e(b)$$

Expected time to reach a state – example



Example. What's the expected number of steps before the message is delivered?

$$T = \{\text{delivered}\}$$

$$e(\text{delivered}) = 0$$

$$e(\text{start}) = 1 + 0.9 \cdot e(\text{error}) + 0.1 \cdot e(\text{delivered})$$

$$e(\text{error}) = 1 + 0.5 \cdot e(\text{start}) + 0.5 \cdot e(\text{error})$$

$$\Rightarrow e(\text{start}) = 28$$

Formal specifications for DTMCs

PCTL = Probabilistic Computation Tree Logic

- tree description of system executions/paths
- probabilistic operator over system paths
- example: *if a message is sent, then it will be delivered within 10 steps with probability at least 95%*

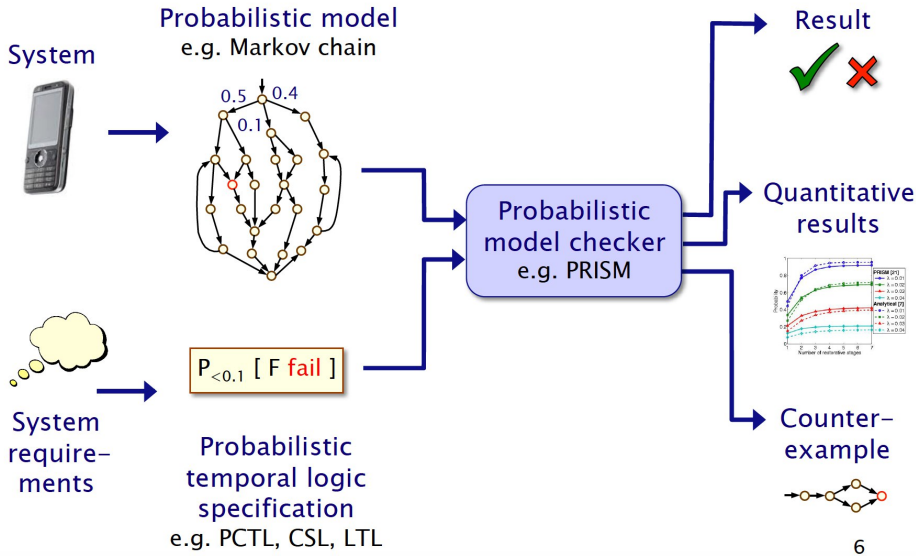
$$\text{send} \Rightarrow P_{\geq 0.95} (F^{\leq 10} \text{ deliver})$$

PLTL = Probabilistic Linear Temporal Logic

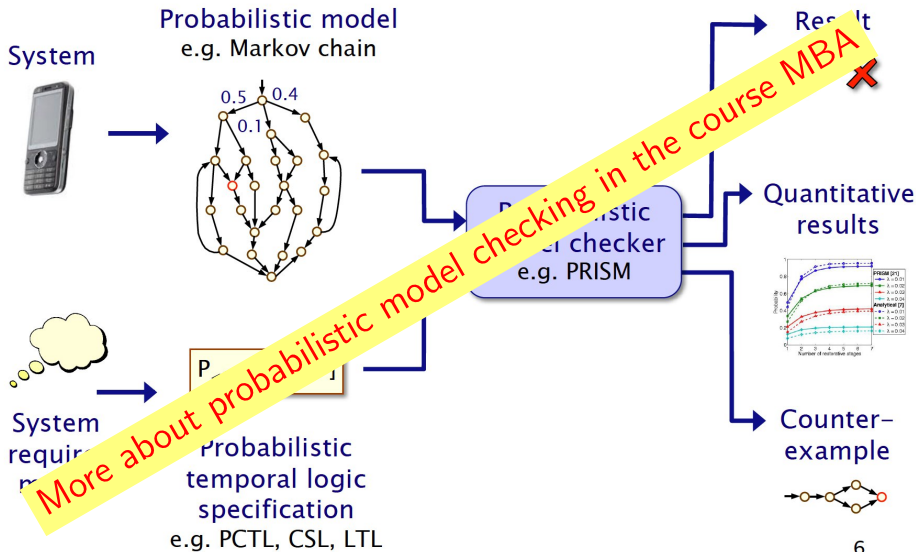
- linear description of system executions/paths
- probabilistic operator over system paths
- example: *with probability at most 1%, an unrecoverable error occurs*

$$P_{\leq 0.01} (\text{FG error})$$

Overview of probabilistic model checking



Overview of probabilistic model checking

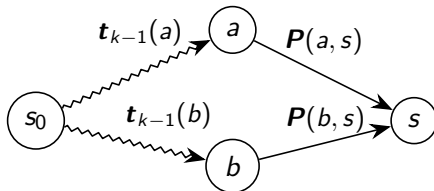


Recap: transient analysis

How does the system evolve in time (after starting in s_0)?

- *transient probability*: $t_k(s) = P(X(k) = s \mid X(0) = s_0)$
 - the probability of being in state $s \in S$ at time $k \geq 0$
- sum probabilities of all paths of length k leading to s (expensive)
- apply the Markov property directly:

$$t_k(s) = \sum_{s' \in S} t_{k-1}(s') \cdot P(s', s) \text{ where } t_0(s_0) = 1$$

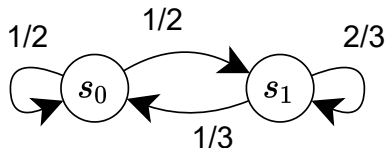


$$t_k(s) = t_{k-1}(a) \cdot P(a, s) + t_{k-1}(b) \cdot P(b, s)$$

Limiting distribution

- What happens to the transient probability distribution $\mathbf{t}_k$ if we let the system evolve for a long time?
- $\mathbf{t}_\infty(s) = \lim_{k \rightarrow \infty} \mathbf{t}_k(s) = \lim_{k \rightarrow \infty} P(X(k) = s \mid X(0) = s_0)$
- If the limit above exists, we say that $\mathbf{t}_\infty$ represents the *limiting distribution*
- $\mathbf{t}_\infty(s)$ corresponds to the long-run frequency of visiting state s

Limiting distribution – example



$$t_0 = (1.00000, 0.00000)$$

$$t_1 = (0.50000, 0.50000)$$

$$t_2 \approx (0.41666, 0.58334)$$

$$t_3 \approx (0.40277, 0.59723)$$

$$t_4 \approx (0.40046, 0.59954)$$

$$t_5 \approx (0.40007, 0.59993)$$

$$t_6 \approx (0.40000, 0.60000)$$

$$t_7 \approx t_\infty = (2/5, 3/5)$$

$$t_0 = (0.00000, 1.00000)$$

$$t_1 \approx (0.33333, 0.66667)$$

$$t_2 \approx (0.38888, 0.61112)$$

$$t_3 \approx (0.39814, 0.60186)$$

$$t_4 \approx (0.39969, 0.60031)$$

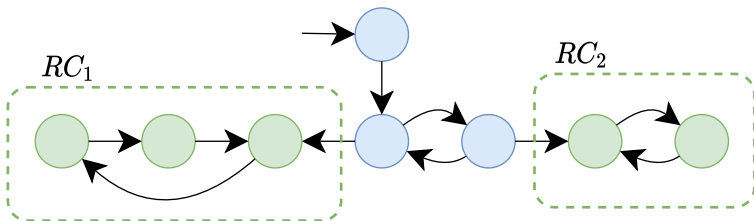
$$t_5 \approx (0.39994, 0.60006)$$

$$t_6 \approx (0.40000, 0.60000)$$

$$t_7 \approx t_\infty \approx (2/5, 3/5)$$

Classification of states

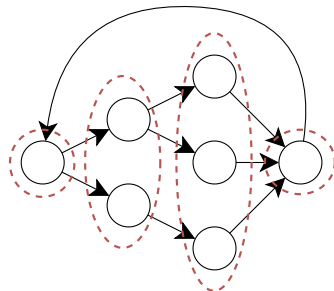
- state s is called *recurrent* if, upon leaving s , it is always possible to return back to s
 - once visited, recurrent states are visited infinitely often
 - states reachable from a recurrent state form a *recurrent class*
 - in terms of graph theory, recurrent class = bottom strongly connected component (BSCC)
- non-recurrent states are called *transient*
 - transient states are eventually abandoned forever



- if the chain has a single recurrent class, then the limiting distribution is independent of the initial state

Periodicity

- Recurrent class is *periodic* if its states can be grouped into *cyclic classes* such that transitions from one class lead to the next one.
- Hint: if at least one state contains a self-loop, the class is aperiodic



Theorem

A chain with aperiodic recurrent classes has a limiting distribution.

Steady state theorem

Theorem (Steady State Convergence Theorem)

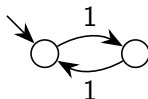
*In a chain with a single aperiodic recurrent class, the limiting distribution is independent of the initial state and coincides with the **steady-state (stationary) distribution** π satisfying:*

- *system of balance equations: $\forall s: \pi(s) = \sum_{s'} \pi(s') \cdot P(s', s)$*
- *normalisation condition: $\sum_s \pi(s) = 1$*

or, using matrix notation, $\pi = \pi \cdot P$, $\mathbf{1}^T \pi = 1$.

Properties:

- $\pi(s) = 0$ for each transient state s
- stationary distribution π always exists, even if the limiting does not, see e.g. the following periodic chain with $\pi = (0.5, 0.5)$



Steady-state analysis – example

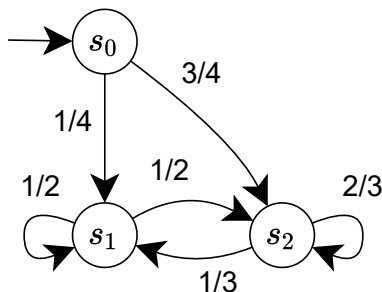
$$\pi(s_0) = 0$$

$$\pi(s_1) = \frac{1}{2}\pi(s_1) + \frac{1}{3}\pi(s_2)$$

$$\pi(s_2) = \frac{1}{2}\pi(s_1) + \frac{2}{3}\pi(s_2)$$

$$\pi(s_1) + \pi(s_2) = 1$$

$$\Rightarrow \pi = \left(0, \frac{2}{5}, \frac{3}{5}\right)$$



Recap: eigenvalues and eigenvectors

- Let $\mathbf{A}$ be a square matrix.
- Column vector $\mathbf{x} \neq \mathbf{0}$ is the (*right*) *eigenvector*¹ of $\mathbf{A}$ associated with *eigenvalue*² $\lambda \in \mathbb{R}$ if

$$\mathbf{Ax} = \lambda \mathbf{x}.$$

- Row vector $\mathbf{y} \neq \mathbf{0}$ is the *left eigenvector* of $\mathbf{A}$ associated with eigenvalue $\lambda \in \mathbb{R}$ if

$$\mathbf{yA} = \lambda \mathbf{y}.$$

- Eigenvector is not affected by the linear transformation $\mathbf{A}$, only scaled by λ .
- Note that if $\mathbf{Ax} = \lambda \mathbf{x}$, then $\mathbf{x}^T \mathbf{A}^T = \lambda \mathbf{x}^T$, i.e. the right eigenvector of $\mathbf{A}$ is the transpose of the corresponding left eigenvector of $\mathbf{A}^T$.

¹vlastní vektor

²vlastní číslo

Steady-state analysis via eigenvectors

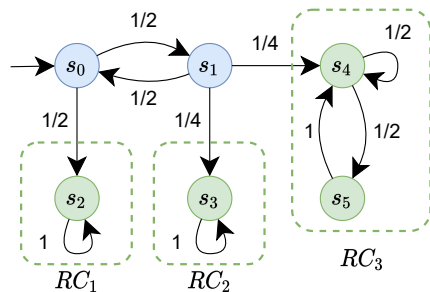
- The steady-state distribution π satisfies

$$\pi P = 1 \cdot \pi.$$

- Therefore, π is the left eigenvector associated with eigenvalue 1 of the probability transition matrix P .
- Conversely, to find the steady-state distribution, find the left eigenvector y of P associated with eigenvalue 1 and normalise y so that $\mathbf{1}^T y = 1$.

Homework. Prove that each stochastic matrix P has a left eigenvector associated with eigenvalue 1, and therefore has a steady-state distribution.

Steady-state analysis – general case



$$P = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 0 & 1/4 & 1/4 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

The chain with multiple (aperiodic) recurrent classes has infinitely many steady states, e.g. all the following distributions satisfy $\pi P = \pi$:

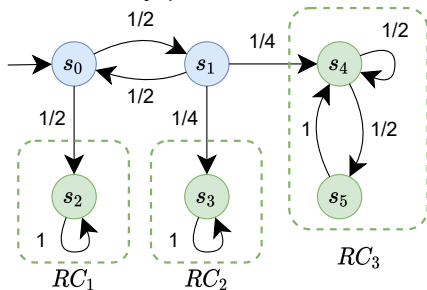
$$\pi_1 = (0, 0, 1, 0, 0, 0)$$

$$\pi_2 = (0, 0, 1/3, 1/3, 2/9, 1/9)$$

$$\pi_3 = (0, 0, 0, 1/2, 1/3, 1/6)$$

Steady-state analysis – general case

To obtain the limiting distribution $\mathbf{t}_\infty$ for a given initial state, compute steady-state distributions of individual recurrent classes and multiply them by the *reachability probabilities*.



$$\pi^{RC_1} = (0, 0, 1, 0, 0, 0)$$

$$\pi^{RC_2} = (0, 0, 0, 1, 0, 0)$$

$$\pi^{RC_3} = (0, 0, 0, 0, 2/3, 1/3)$$

$$P(s_0 \rightarrow RC_1) = 2/3$$

$$P(s_0 \rightarrow RC_2) = 1/6$$

$$P(s_0 \rightarrow RC_3) = 1/6$$

$$\mathbf{t}_\infty = \sum_i P(s_0 \rightarrow RC_i) \cdot \pi^{RC_i} = \left(0, 0, \frac{2}{3}, \frac{1}{6}, \frac{1}{9}, \frac{1}{18}\right)$$

Note: in this case, 1 is a repeated eigenvalue of $\mathbf{P}$ and the limiting distribution $\mathbf{t}_\infty$ is a linear combination of the corresponding left eigenvectors.